

PyDIPS: A Python-Based Dynamic Intrusion Prevention System using Machine Learning for Real-Time Threat Mitigation

Huzaifa and Ammar Asad

BEE17D ICT Course Final Project

School of Electrical Engineering and Computer Science

1. Abstract

The rapid evolution of cyber threats, particularly high-speed Denial-of-Service (DoS) attacks and zero-day exploits, has rendered traditional signature-based security systems inadequate. Legacy firewalls rely on static databases of known malicious patterns, creating a “protection gap” where systems remain vulnerable to novel or slightly altered attack vectors. This paper introduces the Python-Based Dynamic Intrusion Prevention System (PyDIPS), a host-based solution that integrates real-time packet analysis with automated machine learning classification to bridge the gap between passive detection and active prevention.

Utilizing the Scapy library for low-level packet sniffing, PyDIPS extracts critical behavioural features including packet size, time interval and destination port. These features serve as the input for a Random Forest Classifier, which was trained

on a comprehensive database of over 42,000 network events to distinguish between benign traffic bursts and malicious intrusions. Experimental results demonstrate that PyDIPS classification accuracy of 98.54 %.

Beyond Detection, the system implements a dynamic mitigation engine that programmatically interacts with the Windows Advanced Firewall to block identified threats instantly. To address the challenge of false positives, PyDIPS incorporates a consecutive packet threshold logic, ensuring that only sustained malicious behaviour triggers a response. Furthermore, the system employs a self-healing unblocking mechanism using threaded timers, allowing for temporary quarantine of suspect Ips and maintain network availability. This research demonstrates that an AI-driven, host-based approach provides a scalable and highly effective defence against automated network threats.

2. Introduction

A. The crisis of traditional security,

The modern threat landscape is dominated by automated scripts and “low-and-slow” intrusions that easily bypass traditional defences. Legacy systems like standard firewalls rely on a database of known malicious signatures. This reactive approach creates a “protection gap” where a system remains vulnerable until a new signature is manually developed and deployed.

B. System Limitations,

- **Novel Attack Vulnerability:**

Any malware variant that deviates even slightly from an established signature will bypass detection entirely.

- **Security Fatigue:**

Overly rigid rules lead to false positives, overwhelming administrators with alerts and causing legitimate traffic to be dropped.

- **Reactive Delay:**

Most Intrusion Detection Systems (IDS) log attacks only after they have

occurred, rather than preventing them in real-time.

C. The PyDIPS Objective,

PyDIPS is designed to be a “proactive” system rather than a “reactive” system. By focusing on traffic behaviour (packet frequency, size and destination address and port) rather than some specific content, the system can identify and block threats that have never been seen before.

3. Methodology and System Architecture

PyDIPS is built upon a three-layered software stack that integrates networking, data science and system control.

A. Data Acquisition (Networking Layer),

The first phase involves capturing raw network traffic to build a training dataset. The system uses **Scapy**, an industry-standard library, to sniff packets directly from the network interface.

1. The process_packet Logic:

Every incoming TCP/IP packet is dissected to extract three primary features; packet size, time interval (the duration between the current

and previous packet), and the destination port.

2. **Data Persistence:** These features are stored in a CSV database (stored as traffic data.csv) using the **Pandas** library, which allows for rapid feature engineering and model training.

B. Machine Learning (Data Science Layer),

The core of the detection engine is a **Random Forest Classifier** implemented via **Scikit-learn** library (Used to train various model and create training and testing data from a database).

1. **Training:** The model is trained on a labelled data where 1 represented an attack and 0 represented normal traffic.
 2. **Classification:** Once trained, the model evaluates live traffic. It assigns a probability score to each packet flow; if the confidence exceeds 95% and the packet interval is extremely low (less than 0.05 seconds)), it is flagged as an active threat.
-

4. Technical Implementation: Code Breakdown

To understand the "Technical Depth" of the innovation, we must analyse the interaction between the Python script and the Operating System.

A. Real-Time Prevention Logic,

In the prevention script, the system loads the pre-trained model using joblib. As packets arrive, the process_packet function serves as the central hub:

1. **IP filtering:** The function firsts checks if the Source IP is already in a Blocked_Ips list to avoid redundant processing (refer to code below).

```
if src_ip in Blocked_Ips:  
    return
```

2. **Behavioural Check:** It measures the time_interval. High-speed attacks often have intervals near zero (refer to code below).

```
Time_interval = current_time -  
                packet_time
```

3. Burst Counter: A

Burst_threshold is set (defaulting to 5 packets). This ensures that a single outlier packet does not trigger a ban, but a sustained burst of suspicious activity will (refer to code below).

```
Burst_threshold = 5
attack_counter = 0
# ...
if attack_counter >= Burst_threshold:
```

B. The Automated Blocking Mechanism (block_ip),

When the threshold is reached, the block_ip function is called. This function uses the subprocess module to execute system-level commands:

1. Command:

```
Command = f"netsh advfirewall firewall
            rule name=... ..."
```

2. **Action:** It creates a unique rule(e.g. Block_AI_10.7.40.1) that instructs the Windows Firewall to "DROP" all incoming traffic from that specific IP address.

C. Self-Healing Mechanism (unblock_ip),

A critical innovation in PyDIPS is the use of threading.Timer. To prevent permanent lockouts of legitimate users (false positives), the system automatically schedules an unblocking event 30 seconds after the initial block. This allows the system to remain "dynamic" and "adaptive" without requiring manual administrative intervention (refer to code below).

```
timer = threading.Timer(Block_time,
                        sunblock_ip, args=[ip_address...])
timer.start()
```

5. Opportunities and Challenges

A. Opportunities for Implementation,

PyDIPS is highly scalable for individual workstations and IoT devices that cannot afford expensive hardware-based IPS solutions. Because it is written in Python, the logic can be easily ported to Linux (using iptables) or integrated into cloud environments (using AWS Security Group APIs).

B. Technical Challenges,

1. **Resource Overheard:** Real-time sniffing and ML prediction for every packet can introduce latency on high-traffic servers.
2. **Encrypted Traffic:** While PyDIPS looks at packet headers, it cannot inspect encrypted payloads. However, its reliance on timing and size metadata allows it to remain effective even against HTTPS or SSH- based attacks.

6. Ethical Perspective and Societal Impact

A. Impact on Privacy and Safety,

The implementation of PyDIPS provides a layer of security that protects personal data from unauthorized access. By automating the defence process, it ensures that systems remain protected even when a human administrator is not present, all while being about as accurate as one.

B. Ethical Dilemmas in Automated Blocking,

Automating the firewall introduces ethical risks regarding "False Positives". If the AI incorrectly blocks a critical service

(such as a medical data feed or a financial transaction), the impact could be severe. This highlights the need for:

1. **Transparency:** Maintaining a detailed monitoring log for post-incident auditing by a human operator.
2. **Accountability:** Establishing clear human-in-the-loop protocols for reviewing the AI's decisions, thus ensuring that not every part of the program is in control of the AI.

7. Results and Future Work

During rigorous internal testing phases, the PyDIPS framework demonstrated high efficacy by successfully identifying simulated Denial-of-Service (DoS) attack vectors totalling 220 malicious packets. Most notably, the system triggered a proactive firewall block within just five packets of the initial attack onset, effectively neutralizing the threat before significant resource exhaustion occurred. The underlying Random Forest classification model maintained a consistent accuracy rate exceeding 95%, which was achieved by meticulously correlating destination port targeting with high-

frequency packet arrival intervals. This statistical approach allowed the system to distinguish between legitimate burst traffic and coordinated malicious surges with high precision. Future work will focus on:

A. Multi-Threading,

Architecting a parallel processing engine to significantly enhance the sniffer's throughput. This optimization aims to minimize packet drop rates and ensure the system can maintain real-time monitoring capabilities even when handling high-capacity Gigabit network speeds.

B. Deep Learning,

Transitioning the detection core from traditional Random Forest algorithms to more sophisticated Recurrent Neural Networks (RNN). By leveraging Long Short-Term Memory (LSTM) cells, the system will be better equipped to analyse complex, long-term traffic sequences and identify subtle temporal patterns that precede sophisticated multi-stage attacks.

8. Conclusion

PyDIPS demonstrates that integrating Machine Learning with system-level controls offers a powerful defence against modern

cyber threats. By moving away from static signatures and embracing dynamic behavioural analysis, we have created a system that is not only smarter but significantly faster than traditional security measures.

9. References

A. Scapy Documentation,

For real-time packet manipulation
(<https://scapy.readthedocs.io/> or <https://scapy.net/>)

B. Scikit-learn,

For the implementation of Random Forest classification
(<http://scikit-learn.org/>).

C. Microsoft Netsh Documentaion,

For Windows Firewall automation
(<https://learn.microsoft.com/en-us/windows-server/administration/windows-commands/netsh>).

D. BEE-17-D ICT Project Proposal,

By Muhammad Huzaifa and Ammar Asad Warraich
(<https://drive.google.com/file/d/1YYrUg7mciqFOYVB83poEITX0PFULxQJA/view?usp=sharing>).

10. Appendix

The complete Python implementation for the AI-driven network defence system, including the packet sniffing logic and firewall integration, can be accessed via the link below.

[\(Source Code file, Model File and Database Used\)](#)

Or use the QR code below to access the same files as mentioned,

